Programação Orientada a Objetos - 2026/2027

Trabalho Prático

O trabalho prático de POO consiste na elaboração do programa aqui descrito. O programa deve:

- Ser feito na linguagem estudada nas aulas, usando corretamente a sua semântica e os princípios de orientação a objetos;
- Usar as classes/bibliotecas *standard* da linguagem abordadas nas aulas, onde apropriado e permitido;
- Não devem ser usadas outras bibliotecas sem o consentimento prévio dos docentes;
- Deve concretizar as funcionalidades referidas no enunciado;
- Não é permitida uma abordagem baseada na mera colagem de excertos de outros programas, de exemplos ou outras fontes. Todo o código apresentado terá que ser demonstradamente entendido por quem o apresenta e explicado em defesa, caso contrário, o trabalho não será contabilizado.

Visão geral

O trabalho consiste num simulador de fábrica em Marte (para não poluir a Terra). A área da fábrica é um retângulo visto como uma grelha. Em cada posição pode haver apenas solo (vazio), uma cratera, ou um depósito de matéria-prima (ferro, silício ou gelo). Sobre o solo e sobre os depósitos podem ser colocadas máquinas.

O utilizador do simulador é o gestor da fábrica. Dispõe de uma quantia de créditos com a qual compra máquinas, coloca-as em posições da área da fábrica, orientadas numa determinada direção, e liga-as entre si de forma a criar linhas de produção: as matérias-primas são retiradas dos depósitos por perfuradoras, levadas por transportadores, transformadas em refinarias e oficinas, e finalmente enviadas para a Terra a partir de um lançador, o que rende mais créditos. O gestor pode também melhorar algumas máquinas e contratar engenheiros que aceleram a produção. Os detalhes da funcionalidade são descritos mais adiante.

O simulador incluirá a simulação da passagem do tempo. A passagem do tempo é independente do tempo real medido pelo computador, sendo descrita em unidades abstratas de “instantes”, e avança-se para o instante seguinte por ordem do utilizador. A cada instante que passa, as máquinas atuam de acordo com a sua natureza: perfuram, transportam, transformam, armazenam ou enviam itens.

A simulação termina quando o utilizador entender: não é um jogo com objetivos para ganhar ou perder.

Interação com o utilizador

Ecrã

O programa destina-se ao ambiente consola, onde apenas se consegue imprimir “para a frente e para baixo”: não existem, na biblioteca *standard*, funções para posicionar o cursor numa linha e coluna do ecrã, limpar o ecrã ou desenhar em posições arbitrárias. Para ultrapassar esta limitação, toda a apresentação visual da fábrica e do painel de estado deve ser feita através de uma classe utilitária “Ecra”, desenvolvida pelos alunos, que representa o conceito de ecrã de computador (tipo consola). Esta classe tem as seguintes características:

- Guarda internamente os caracteres do ecrã num buffer interno que é um array bidimensional (não pode ser usada, nesta classe, nenhuma coleção) com dimensões máximas predefinidas de 40 linhas por 80 colunas, podendo ser usadas menos que esses valores máximos (desperdiçando-se alguma memória).
- Disponibiliza funções de alto nível, tais como: limpar o “ecrã” (o seu buffer interno, colocando espaços); posicionar o cursor (virtual) numa linha e coluna, sendo a escrita (no seu buffer interno) seguinte feita a partir dessa posição; escrever (no seu buffer interno) um caracter ou uma cadeia de caracteres numa linha

e coluna; desenhar uma moldura retangular dada a linha, coluna, altura e largura; e outras que se considerem úteis.
- O conteúdo só aparece na consola quando for pedido explicitamente (por exemplo, através de uma função mostra), sendo nesse momento enviado de uma só vez (todas as linhas do buffer interno).
- Escritas fora dos limites são ignoradas (cortadas), ou passam para a linha de baixo, nunca causando erros.
- A classe é um utilitário genérico: trabalha com caracteres, não tendo qualquer conhecimento sobre fábricas, máquinas ou comandos, podendo ser reutilizada, sem alterações, noutro programa.

Neste enunciado, o termo "ecrã virtual" refere-se ao buffer interno que existe dentro de um objeto desta classe.

O utilizador irá interagir com o simulador através de comandos escritos. Esses comandos, descritos mais adiante, permitem comprar e configurar máquinas, gerir engenheiros, listar informação, passar para o instante seguinte, e outras operações de controlo.

Representação da fábrica

A área da fábrica (ou simplesmente "fábrica") é apresentada numa área do ecrã delimitada por uma moldura, um caracter por cada posição. Na interação com o utilizador, a identificação da posição é feita usando letras para referir a linha e a coluna. Por exemplo, a posição na linha 3 e coluna 5 será a posição `ce`; a posição na linha 11 e coluna 3 será a posição `kc`. O canto superior esquerdo corresponde à posição `aa`. Para ajudar o utilizador a identificar as posições, a representação da fábrica deve incluir uma "régua" constituída pelas letras que identificam as linhas e colunas, impressa em maiúsculas para não se confundir com o conteúdo. O tamanho da fábrica é decidido pelo utilizador no início da simulação, nunca mudando até ao fim desta, e pode ser, no máximo, 26x26. A colocação da representação visual da fábrica será grandemente facilitada pelo uso da classe "Ecra" referida atrás, colocando o conteúdo primeiro no ecrã virtual e só depois passando-o para a consola. É importante perceber que a área do ecrã virtual pode ser maior do que o da área da fábrica, para poder apresentar outras informações.

Ao lado da fábrica existe um painel de estado, também com uma moldura, com: o instante atual, os créditos, o número de máquinas, o número de engenheiros e um resumo do que aconteceu no último instante (exemplo na figura abaixo. Os caracteres usados são explicados adiante; a disposição exata fica ao critério de cada grupo):

```
+----------------------+  +--------------------------------+
|  ABCDEFGHIJKLMNOPQRST |  | Instante: 57                   |
|A                 #### |  | Créditos: 412                  |
|B   %%%               |  | Máquinas: 22                   |
|C   %P>f>>R>pv        |  | Engenheiros: 1                 |
|D   %%%     v         |  |                                |
|E           v         |  | No último instante:            |
|F   ~~P>g>R  O>k>L    |  |  enviado k (+120 créditos)     |
|G   ~~~   v           |  |  depósito esgotado em hr       |
|H         S       P   |  |                                |
|                      |  |                                |
+----------------------+  +--------------------------------+
```

O ecrã virtual deve ter tamanho suficiente para conter a área da fábrica e o painel. As respostas dos comandos de listagem, que podem ser longas, não precisam passar pelo ecrã virtual: podem ser escritas diretamente na consola, debaixo da área da fábrica e painel.

Atenção: Ao longo do texto, vão sendo indicados valores numéricos para vários parâmetros. Os valores apresentados são apenas exemplos. **Os valores encontram-se especificados numa classe Settings (fornecida pelos docentes) que deve ser usada**. Durante a apresentação/defesa, poderá ser pedida uma alteração a estes valores e o programa deverá assumir os novos valores.

Conceitos principais envolvidos e detalhes

Área da Fábrica

É constituída por um conjunto de posições com uma forma retangular. Cada posição tem um terreno de um dos seguintes tipos:

- Solo: posição normal, onde pode ser colocada qualquer máquina. Quando vazia, é representada por um espaço.

- Cratera: posição onde não pode ser colocado nada. Representada por #.
- Depósito: posição que contém uma quantidade finita de uma matéria-prima (minério de ferro, silício bruto ou gelo). Cada uma destas posições pode conter qualquer matéria-prima, mas apenas uma delas, determinada aleatoriamente entre ferro, silício ou gelo, se a fábrica for criada com o comando fabrica, ou indicada no ficheiro do mapa, se for criada com o comando carrega. A quantidade inicial de matéria-prima é também determinada aleatoriamente, com um valor entre 40 e 60 unidades. Pode ser colocada qualquer máquina num depósito, mas apenas uma perfuradora tira proveito dele. Quando esgota, a posição passa a ser solo. Representação: depósito de ferro %, depósito de silício $, depósito de gelo ~.

Em cada posição pode existir, no máximo, uma máquina. A fábrica tem uma quantia de créditos (inicialmente 1000), que é usada nas compras e aumenta com os envios para a Terra, e um conjunto de engenheiros (explicado adiante).

A área da fábrica pode ser criada de duas formas: indicando apenas as dimensões, sendo, nesse caso, colocados automaticamente e em posições aleatórias as crateras e os depósitos (em número indicado na classe Settings), ou lendo um ficheiro de texto que descreve o mapa (formato indicado mais adiante).

Itens

Os itens são as coisas que circulam pela fábrica: as matérias-primas e os produtos refinados que resultam da sua transformação. Cada item tem um nome, um caracter que o representa e um valor em créditos. Os itens não estão necessariamente todos na mesma categoria de coisas: uma matéria-prima não está a par com um produto refinado, por exemplo. Independentemente deste aspeto, por simplicidade narrativa, a palavra **item** refere-se genericamente às coisas que circulam pela fábrica. Existem os seguintes itens:

Item	Caracter	Valor	Obtido de
Minério de ferro	f	2	Perfuradora em depósito %
Silício bruto	s	3	Perfuradora em depósito $
Gelo	g	1	Perfuradora em depósito ~
Placa de ferro	p	8	Refinaria
Silício puro	i	10	Refinaria
Água	a	4	Refinaria
Kit de habitat	k	120	Oficina

Os itens não estão “soltos” na fábrica: um item encontra-se sempre dentro de uma máquina (num transportador, numa refinaria, num silo, etc.).

Máquinas

Existem vários tipos de máquinas. Todas as máquinas partilham algumas características, mas o seu comportamento varia consoante o tipo. Todas as máquinas:

- Ocupam uma posição da área da fábrica e têm um custo de compra. Não se movem, permanecendo fixas no local onde foram colocadas. Não podem ser colocadas em cima de crateras.
- Operam sobre itens (extraem matérias-primas, transformam matérias-primas em produtos refinados, transportam itens, etc.). Num dado instante, as máquinas podem ter 0, 1 ou mais itens, dependendo do seu tipo. Enquanto permanecem na máquina, os itens pertencem a essa máquina.
- Têm uma **direção de saída**: cima, baixo, esquerda ou direita (c, b, e, d). É para a posição vizinha nessa direção que a máquina tenta entregar os itens que produz ou transporta. A entrega só acontece se nessa posição existir uma máquina e se essa máquina aceitar o item nesse momento. Caso contrário, o item fica na máquina de origem, que poderá ficar bloqueada à espera de o conseguir entregar. Cada máquina decide que itens aceita e em que condições; as máquinas recebem itens vindos de qualquer lado.
- Podem ser vendidas, recebendo a fábrica 50% do custo de compra. Os itens que estiverem dentro da máquina perdem-se.
- **Um item avança no máximo uma posição por instante.** Por exemplo, numa fila de transportadores, um item passa de um transportador para o seguinte a cada instante, e não percorre a fila toda num único

instante, seja qual for a ordem pela qual as máquinas são processadas. Outro exemplo: um item que tenha sido extraído (ou acabado de ser produzido), só no instante seguinte será entregue à máquina adjacente.

No trabalho existem os seguintes tipos de máquina:

Perfuradora

- Se tiver uma unidade de matéria-prima por entregar, tenta entregá-la à máquina adjacente (se houver) na sua direção de saída. Não aceita itens vindos de outras máquinas.
- Se não tiver nenhum item por entregar (guarda no máximo 1), a cada 3 instantes retira 1 unidade de matéria-prima do terreno onde está, se este for um depósito não esgotado. Fora de um depósito não extrai nada.
- Existe uma probabilidade de 40% do item extraído se perder. Este assunto pode ser melhorado pelo gestor, diminuindo a probabilidade em 10% mediante o pagamento de 15 créditos. Pode ser melhorado às vezes que o gestor quiser, sempre 10% de cada vez e com o mesmo custo, até a probabilidade chegar a 0%.
- Custo: 100. Representada por P.

Transportador

- Transporta 1 item de cada vez. Aceita itens de qualquer tipo, desde que esteja vazio.
- Em cada instante, tenta entregar o item que transporta na sua direção de saída.
- Custo: 5. Quando vazio, é representado pela seta correspondente à sua direção (^ v < >). Quando transporta um item, é representado pelo caracter desse item (permite ver os itens a circular).

Refinaria

- Transforma matérias-primas em produtos refinados: minério de ferro em placa de ferro, silício bruto em silício puro e gelo em água. Cada matéria-prima é que sabe em que se transforma.
- Aceita apenas matérias-primas (minério de ferro, silício bruto e gelo), e apenas 1 unidade de cada vez, e apenas se estiver vazia.
- A transformação começa assim que recebe a matéria-prima e demora 4 instantes. No fim, fica com o produto refinado, que tenta entregar, no instante seguinte, na sua direção de saída. Enquanto não o conseguir entregar, não aceita nova matéria-prima.
- Custo: 150. Representada por R.

Oficina

- Produz kits de habitat segundo uma receita fixa: 2 placas de ferro + 1 silício puro + 1 água → 1 kit de habitat. Aceita apenas estes itens, até ter a quantidade necessária de cada um, e inicia a produção assim que os tem todos. Reconhece os ingredientes comparando-os com a receita. A produção demora 5 instantes. A velocidade de produção pode ser melhorada pelo gestor, diminuindo 1 instante a duração da produção, pagando 30 créditos por isso. Esta melhoria só pode ser feita uma vez.
- Quando conclui a produção, fica com o item produzido até que o consiga entregar à máquina adjacente na sua direção de saída. Enquanto tiver um item produzido por entregar, não aceita nenhum item novo.
- Custo: 200. Representada por O.

Silo

- Máquina simples que guarda itens de qualquer tipo, sem limite de capacidade. Serve para acumular itens quando a máquina seguinte não os consegue receber ao ritmo a que chegam. Em cada instante, há uma probabilidade de 20% de se perder o item guardado há mais tempo. A probabilidade pode ser melhorada pelo gestor em 5% de cada vez, pagando 50 créditos por melhoria, até chegar a 0%.
- Em cada instante, tenta entregar na sua direção de saída o item que se encontra guardado há mais tempo.
- Custo: 120. Representado por S.

Lançador

- É uma espécie de catapulta eletromagnética que projeta itens na direção da Terra. Aceita qualquer item. Assume-se imediatamente a venda do item como efetuada, sendo o valor correspondente somado aos créditos da fábrica. A sua direção de saída é irrelevante. O disparo da catapulta requer a acumulação de muita energia e só consegue enviar um item de 4 em 4 instantes. Enquanto tiver um item por lançar, não aceita novos itens.

- Existe uma probabilidade de 30% do item se perder no lançamento (o crédito da sua venda é perdido). Este assunto pode ser melhorado pelo gestor, diminuindo a probabilidade em 10% mediante o pagamento de 25 créditos. Pode ser melhorado às vezes que o gestor quiser, sempre 10% de cada vez e com o mesmo custo, até a probabilidade chegar a 0%.
- Custo: 80. Representada por L.

Engenheiros

A fábrica pode ter engenheiros. Cada engenheiro é contratado pelo gestor, pagando 50 no momento da contratação, e recebe depois um salário de 1 crédito por instante. Cada engenheiro tem um nome, definido na sua criação, e um número identificador, atribuído automaticamente, sendo o primeiro o número 1.

Um engenheiro pode estar livre ou estar associado a uma máquina. Enquanto está associado a uma máquina, altera o comportamento desta, como se descreve:

- Perfuradora, Refinaria, Oficina, Lançador: a duração da operação da máquina diminui em 1 unidade por engenheiro, nunca sendo inferior a 1. Por exemplo, se houver três engenheiros associados a um lançador, o tempo de recarga entre dois lançamentos baixa 3, ficando em 1. Engenheiros adicionais neste lançador já não terão efeito.
- Transportador e Silo: os engenheiros não têm efeito, mas podem ser associados na mesma.

Uma máquina pode ter vários engenheiros associados, mas cada engenheiro só pode estar associado a uma máquina. A associação muda através de comandos: atribui passa o engenheiro para outra máquina; despede recambia-o para a Terra, deixando de fazer parte da fábrica (e de receber salário); e, se a máquina for vendida, os engenheiros que lhe estavam associados ficam livres. Se os créditos ficarem a 0 ou abaixo disso, todos os engenheiros abandonam a fábrica.

Funcionamento e uso do simulador

A simulação decorre por turnos (instantes), que avançam por ordem do utilizador. Em cada novo instante, as máquinas atuam, os salários dos engenheiros são pagos, e o simulador pede mais comandos ao utilizador, mostrando o resultado.

O utilizador é livre de mandar executar os comandos que entender antes de avançar para o instante seguinte. As consequências visíveis dos seus comandos devem ser imediatamente representadas.

Em cada novo instante, o simulador apresenta o ecrã atualizado: a fábrica e o painel de estado com o resumo do que se passou (máquinas que fizeram a sua função ou pararam por estar cheias, itens enviados e créditos obtidos, depósitos esgotados). O utilizador pode solicitar informações adicionais, existindo comandos para o efeito.

As ordens do utilizador são dadas textualmente, segundo o paradigma dos comandos escritos. Cada ordem é escrita como uma frase em que a primeira palavra é um comando e as palavras seguintes são os parâmetros ou informações adicionais. **A linha de texto correspondente à ordem é escrita na totalidade** (não há perguntas subsequentes), só então sendo interpretada e executada. O programa deve validar a sintaxe e coerência do comando (toda a informação/parâmetros necessários foram escritos? Os valores que eram supostos ser inteiros são mesmo inteiros? Estão pela ordem certa?). **Pode ser assumido que será sempre tudo escrito em minúsculas.**
Cada comando é responsável por validar os seus parâmetros, executar-se e gerar a sua própria ajuda.

Comandos do utilizador

Na descrição dos comandos:

- <n> refere-se a um valor (número ou texto), e os caracteres **<** e **>** não fazem propriamente parte do que deve ser escrito.
- < x | y | z> indica que deve ser indicado apenas um dos x, y, z (novamente, sem os **<>**, nem **|**).
- [v] indica que o parâmetro v é opcional, e os caracteres [e] também não devem ser escritos.
- <lc> refere-se a uma posição da fábrica, indicada por duas letras (linha e coluna).
- <dir> refere-se a uma direção: c, b, e ou d (cima, baixo, esquerda, direita).

Nota: Todos os comandos que alteram o estado da fábrica devem causar nova apresentação do ecrã. Em todas as ações que não sejam concretizadas por não estarem reunidas as condições (por exemplo, comprar sem créditos suficientes), o utilizador deve ser informado adequadamente.

Comandos de criação da fábrica

Um destes dois comandos será obrigatoriamente o primeiro a ser executado com sucesso (exceto o `executa` e o `ajuda`) e, depois disso, nenhum destes dois será aceite novamente. Por razões óbvias, nenhum dos outros comandos pode ser executado (exceto o `executa`, o `ajuda` e o `fim`) sem que a fábrica tenha sido criada.

`fabrica <nl> <nc>` *(exemplo:* `fabrica 12 20`*)*
Cria a fábrica com o número de linhas e de colunas indicado (máximo 26 cada). As crateras e depósitos são colocados em posições aleatórias, nas quantidades indicadas em Settings.

`carrega <nome-do-ficheiro>` *(exemplo:* `carrega mapa1.txt`*)*
Cria a fábrica a partir do mapa descrito no ficheiro indicado. Se o ficheiro não existir ou não respeitar o formato, o comando falha e a fábrica não é criada.

Comando para o tempo simulado

`avanca [n]` *(exemplo:* `avanca 10`*)*
Avança n instantes (valor inteiro positivo; assume o valor 1 se não for indicado), um de cada vez. Em cada instante é despoletado o comportamento das máquinas, e são feitas as contas aos custos e ganhos. O ecrã é apresentado no fim de cada instante.

Comandos para as máquinas

`compra <tipo> <lc> <dir>` *(exemplo:* `compra t ce d`*)*
Compra uma máquina do tipo indicado e coloca-a na posição lc com a direção de saída dir. O tipo é indicado por um caracter: p (perfuradora), t (transportador), r (refinaria), o (oficina), s (silo), l (lançador). Falha se a posição estiver ocupada, ou for uma cratera, ou se não houver créditos suficientes.

`vende <lc>` *(exemplo:* `vende ce`*)*
Vende a máquina que se encontra na posição indicada.

`roda <lc> <dir>` *(exemplo:* `roda ce b`*)*
Muda a direção de saída da máquina que se encontra na posição indicada.

`melhora <lc>` *(exemplo:* `melhora hi`*)*
Melhora a máquina que se encontra na posição indicada, de acordo com a natureza da máquina e pagando o respetivo custo. Se a máquina não for melhorável, nada acontece.

Comandos para os engenheiros

`contrata <nome>` *(exemplo:* `contrata joaquim`*)*
Contrata um novo engenheiro com o nome indicado, que fica livre. Falha se não houver créditos suficientes. O nome pode ser repetido, mas o ID é gerado automaticamente e é único. Não há limite quanto ao número de engenheiros, mas cada um recebe o seu salário.

`atribui <id> <lc>` *(exemplo:* `atribui 2 hr`*)*
Atribui o engenheiro com o número indicado à máquina que se encontra na posição lc. Se o engenheiro estava atribuído a outra máquina, deixa de estar e passa para a nova máquina.

`despede <id>` *(exemplo:* `despede 2`*)*
Envia o engenheiro indicado em férias permanentes para a Terra. Deixa de fazer parte da fábrica e de receber salário.

Comandos para listar informação

estado

Apresenta o instante atual, os créditos, as máquinas (tipo, posição, direção, engenheiros associados e resumo do conteúdo), os engenheiros (número, nome, e se está livre ou a que máquina está associado), e o total de itens enviados com sucesso para a Terra, de cada tipo, desde o início da simulação.

lposicao <lc> [n] *(exemplo:* `lposicao df 2`*)*

Mostra a informação da posição indicada: tipo de terreno (e a quantidade de matéria-prima, se for um depósito) e toda a informação da máquina que lá se encontrar, incluindo os itens no seu interior e a sua configuração específica (melhorias, progresso da operação em curso, etc.). Se for indicado o valor n, mostra a informação de todas as posições dentro do quadrado de raio n centrado em lc.

Comandos adicionais de carácter geral

grava <nome> *(exemplo:* `grava antes`*)*

Salvaguarda **em memória** uma cópia da fábrica e de tudo o que nela se encontrar (terreno, máquinas e respetivos itens e estado, créditos, instante, engenheiros). A associação engenheiros-máquinas não fica memorizada na cópia. A cópia fica associada ao nome indicado, podendo ser recuperada mais tarde. Após a cópia feita, a fábrica continua a ser usada normalmente. Se já existir uma cópia com o mesmo nome, é substituída (a anterior é eliminada).

recupera <nome> *(exemplo:* `recupera antes`*)*

Recupera a cópia da fábrica indicada. A fábrica passa a ser como era no momento em que a cópia agora recuperada foi feita, exceto: os engenheiros são recuperados, mas ficam livres. A cópia **não é** eliminada.

executa <nome-do-ficheiro> *(exemplo:* `executa teste1.txt`*)*

Lê comandos a executar do ficheiro indicado. Existe um comando por linha, exatamente com o mesmo formato dos comandos escritos pelo utilizador. Aliás, o processamento dos comandos deve ser o mesmo independentemente da origem destes (teclado ou ficheiro): só muda a origem do texto.

ajuda [comando] *(exemplo:* `ajuda compra`*)*

Sem parâmetro: apresenta a lista de todos os comandos, um por linha, com a sua sintaxe resumida. Com o nome de um comando: apresenta a ajuda detalhada desse comando: sintaxe, significado de cada parâmetro, efeito, condições em que falha, e um exemplo.

fim

Encerra o programa. Isto implica uma saída "limpa", ou seja, libertar todos os recursos alocados.

Implementação dos comandos

A implementação dos comandos deve recorrer a uma hierarquia polimórfica em que cada comando é representado pela sua própria classe. A ação do comando deve ser concretizada por um método polimórfico executa() cujos parâmetros serão aquilo que for necessário para os comandos (no geral) conseguirem efetuar a sua ação. A classe base deve ter o nome Comando.

Formato do ficheiro de mapa

A primeira linha contém o número de linhas e o número de colunas. Seguem-se tantas linhas de texto quanto o número de linhas indicado, cada uma com exatamente o número de colunas indicado, usando os seguintes caracteres: . solo, # cratera, % depósito de ferro, $ depósito de silício, ~ depósito de gelo. Exemplo:

```
6 10
..%%......
..%%...##.
.......##.
~~........
~~....$$..
......$$..
```

Classe de configuração

Todos os valores referidos no enunciado (exemplos: créditos iniciais, custo de cada máquina, probabilidades de perda, salário dos engenheiros, dimensões máximas do ecrã, etc.) devem ser obtidos a partir das constantes que se encontram já definidas na classe `Settings`. Os nomes das constantes são auto-explicativos. Esta classe é fornecida, estando o seu código no ficheiro que acompanha este enunciado. O uso desta classe é obrigatório. Os valores das constantes podem ser alterados para efeitos de testes, mas devem ser repostos na versão entregue. As probabilidades de perda (perfuradora, silo e lançador) também estão definidas em Settings, podendo ser colocadas a 0 para tornar a simulação determinística (por exemplo, para testes).

Requisitos quanto à implementação

- O programa deve compilar sem qualquer warning. Deve executar sem qualquer erro ou exceção não tratada. O código deve ser robusto e completo.
- É esperado um programa verdadeiramente orientado a objetos. Não o sendo, terá uma cotação de 0 ou muito próxima disso.
- Toda a apresentação visual da fábrica e do painel de estado passa pela classe que representa o ecrã, descrita atrás.
- A implementação dos comandos deve passar pelo uso do polimorfismo, como descrito atrás.
- A implementação do armazenamento de itens do silo não pode recorrer a nenhuma coleção da biblioteca *standard*. Não deve ser usada mais memória do que a realmente necessária para os itens que o silo contém em cada momento.

Importante: O enunciado é longo para tentar responder a perguntas que ocorrem frequentemente. Quantidade de texto não significa quantidade de trabalho. Pode haver omissões que podem ser resolvidas com recurso à matéria das aulas e ao bom senso. Na resolução, não podem ser tomados atalhos que removam matéria ou contrariem o enunciado. Em caso de dúvida, devem abordar os docentes nas aulas.

Regras gerais do trabalho

As que estão descritas na FUC e nos *slides* da primeira aula teórica:

- Grupos de dois alunos no máximo, sem exceção, do mesmo laboratório.
- Em todas as entregas: projeto CLion e relatório num arquivo zip com o nome indicado mais adiante. O ficheiro executável **não** deve ser incluído no arquivo zip.
- Apresentação/defesa obrigatória em ambos os checkpoints e no final. Ambos os alunos devem comparecer ao mesmo tempo e quem faltar fica sem nota associada a esse checkpoint.

Metas e entregas

Checkpoint 1 → 8 de novembro, 12h00

Requisitos do checkpoint 1:

- Classe que representa o ecrã, completa, com todas as funcionalidades descritas.
- Criação da fábrica: funcionalidade que mais tarde será integrada nos comandos fabrica e carrega, e respetiva apresentação através do ecrã (molduras, régua, crateras e depósitos, painel de estado ainda que com informação parcial).
- Funcionalidade que mais tarde será integrada no comando lposicao (apenas para o terreno) e fim.
- Planeamento das classes: que classes existem, qual o relacionamento entre elas, qual o seu objetivo e quais as funções diretamente relacionadas com o objetivo da classe. As classes que existirem devem já estar organizadas em .h e .cpp.

- Relatório – Simplificado, mas deve incluir a descrição das opções tomadas e as classes planeadas: quais, para que servem / o que representam e qual a relação entre elas. Máximo: 5 páginas (a capa e o índice não contam).

Entrega -> Arquivo zip (não é rar nem arj - é zip), contendo o projeto CLion e o relatório em pdf. O arquivo tem obrigatoriamente o nome seguinte: **poo_2627_m1_nome1_numero1_nome2_numero2.zip**

Checkpoint 2 → 22 de novembro, 12h00

Requisitos do checkpoint 2

Todos os requisitos do checkpoint 1, mais:

- Leitura dos comandos e respetiva validação, tanto do teclado como através do comando executa. Mesmo que nem todos façam ainda algo, todos os comandos devem ser validados quanto à sintaxe e parâmetros, e cada comando deve ser responsável por validar os seus parâmetros e apresentar a sua ajuda.
- Comando ajuda, completo (sem e com parâmetro).
- Comandos fabrica, carrega, compra, vende, roda, avanca, estado e lposicao.
- Silo completo (exceto o comando melhora).
- Mecanismo de atuação das máquinas a cada instante planeado e iniciado. O comportamento completo de cada tipo de máquina é requisito da entrega final.
- Saída limpa com o comando fim, sem recursos por libertar.
- Relatório – atualização do relatório do checkpoint 1, refletindo as classes efetivamente implementadas, as opções tomadas e a participação individual.

Entrega -> arquivo zip, com o projeto CLion e o relatório em pdf, com o nome: p**oo_2627_m2_nome1_numero1_nome2_numero2.zip**

Entrega final → 13 de dezembro, 12h00

Requisitos: programa completo, com todas as funcionalidades descritas no enunciado, incluindo o comportamento completo de todas as máquinas, os engenheiros, o comando melhora e as cópias da fábrica (grava, recupera), com relatório detalhado e completo. No relatório deve incluir uma lista com os requisitos implementados, parcialmente implementados e não implementados (com indicação da razão dos não implementados).

Entrega -> arquivo zip, com o projeto CLion e o relatório em pdf, com o nome: **poo_2627_final_nome1_numero1_nome2_numero2.zip**

Em cada uma das entregas:

- Apenas um dos alunos do grupo faz a submissão e **associa obrigatoriamente a submissão ao colega de grupo**.
- As apresentações serão nas aulas da semana imediatamente seguinte às entregas. Havendo necessidade, serão definidos horários adicionais. No entanto, a presença na aula (laboratório em que estão inscritos) é obrigatória. Os trabalhadores-estudantes devem contactar o docente do laboratório atempadamente para ver a sua situação.
- Não é aceite a funcionalidade do projeto feita antecipadamente. Em cada checkpoint deve ser entregue apenas aquilo que é pedido nesse checkpoint, caso contrário há penalização na nota. Se existir trabalho a mais já realizado, essa funcionalidade deve ser separada e não entregue, e o projeto resultante (e entregue) deve compilar e executar.

A defesa do perfil B será marcada pelo esquema habitual de horários através do nónio.

O não cumprimento dos checkpoints (não fazer a funcionalidade esperada ou não comparecer à apresentação respetiva) acarreta a perda não recuperável da cotação da seguinte forma: checkpoint 1: até 20%, checkpoint 2: até 20% (adicionais), entrega final: 100%.